

CR-001: Student Active/Inactive Status

1. Change Request Details

Field	Details
Change Request ID	CR-001
Title	Student Active/Inactive Status
Priority	Medium
Difficulty	High
Change Type	Database and Full-Stack Enhancement
Affected Version	Student Management System v1.1.0 baseline
Dependency	Existing <code>students</code> table, <code>POST /api/auth/login</code> , existing <code>/api/students</code> endpoints, and the CR-001 dashboard search

2. Business Requirement

ABC Institute needs to manage students whose status changes over time, for example students who have left the institute, completed their studies, or are temporarily not allowed to use the system.

Currently, the only way for an administrator to stop managing a student is to permanently delete the student record. Deletion cannot be reversed, removes the student's information, and increases the risk of losing records by mistake. The administrator also has no way to prevent a student from using the system without deleting the record.

The system should allow administrators to mark a student as Active or Inactive. The record of an inactive student must be kept, but the student must not be able to access the system until the administrator activates the student again.

3. Current Behaviour

- The `students` table stores registration number, name, email, phone number, and timestamps. It has no status column.
- `DashboardPage.jsx` displays Registration No., Name, Email, Phone, and Actions (Edit, Delete). No status is displayed.
- The only way to remove a student from the system is permanent deletion using `DELETE /api/students/:id`.
- `POST /api/auth/login` accepts any user with valid credentials. No student status check exists.
- The `authenticate` middleware verifies only the JWT signature and expiry. An issued token remains valid until it expires (default 1 day), even if the student record changes.
- `StudentDashboardPage.jsx` handles only a 401 response by logging out. Other errors are displayed as text.
- New students are created using `POST /api/students/`. Existing Edit and Delete operations use `student_id`.
- CR-001 client-side search is available on the administrator dashboard.

4. Requested Change

Add an Active/Inactive status to student records and enforce it in the database, backend API, and frontend.

4.1 Database Change

- Add a `status` column to the `students` table: `ENUM('ACTIVE', 'INACTIVE') NOT NULL DEFAULT 'ACTIVE'`.
- All existing students must be `ACTIVE` after the change.
- Update `schema.sql` for new installations.
- Provide a migration script for existing databases. `schema.sql` uses `CREATE TABLE IF NOT EXISTS`, so running it again will not add the column to an existing table.
- Provide a rollback script that removes the column and restores the previous structure.
- Update `seed.sql` to include status values, with at least one `INACTIVE` sample student for testing.
- Update `database/README.md` to document the new column.

4.2 Backend Change

- Return `status` in the student list, student-by-ID, and own-profile responses.
- Add an administrator-only endpoint to change a student's status.
- Reject login of an inactive student after the credentials are verified, with a clear message and HTTP 403.
- Reject protected student requests from an inactive student who holds a previously issued token.
- Create new students as `ACTIVE`. The existing update endpoint must not change status.

The following API behaviour is requested:

Endpoint	Access	Change
POST <code>/api/auth/login</code>	Public	Reject <code>INACTIVE</code> students (403) after credentials are verified.
GET <code>/api/students/</code>	ADMIN	Response includes <code>status</code> .
GET <code>/api/students/:id</code>	ADMIN	Response includes <code>status</code> .
GET <code>/api/students/me</code>	STUDENT	Response includes <code>status</code> . Request is rejected (403) if the student is <code>INACTIVE</code> .
PATCH <code>/api/students/:id/status</code> (new)	ADMIN	Request body { <code>"status": "ACTIVE"</code> } or { <code>"status": "INACTIVE"</code> }. Returns 400 for an invalid value and 404 if the student is not found.
PUT <code>/api/students/:id</code>	ADMIN	No change. Must not modify status.
POST <code>/api/students/</code> , DELETE <code>/api/students/:id</code>	ADMIN	No behavioural change. New students are <code>ACTIVE</code> by default.

4.3 Frontend Change

- Add a `Status` column to the administrator student table, showing an Active or Inactive badge. The status must be shown as text, not by colour alone.

- Add an Activate/Deactivate action for each student. Ask for confirmation before the change, update the table after success, and show an error message on failure.
- Add a status filter (All, Active, Inactive) near the CR-001 search. The filter and the search must work together, and the result count must reflect both.
- If a logged-in student is rejected because the account is inactive, end the session and redirect to the login page. The login page must continue to show the message returned by the server.
- Preserve the existing Add, Edit, Delete, Search, and Clear Search functionality.

5. Scope

In Scope

- `status` column in the `students` table, with default value and migration.
- Migration script, rollback script, and updates to `schema.sql`, `seed.sql`, and `database/README.md`.
- Status in student API responses.
- Administrator-only status update endpoint with validation.
- Login restriction for inactive students.
- Rejection of previously issued tokens for inactive students.
- Status display, Activate/Deactivate action, and status filter on the administrator dashboard.
- Session handling for a student who is deactivated while logged in.
- Regression of CR-001 search and existing CRUD functionality.
- Update of `CHANGELOG.md`.

Out of Scope

- Replacing or changing the existing permanent Delete behaviour.
- Activating or deactivating administrator accounts.
- Additional statuses such as Graduated or Suspended.
- Status history, reason for change, or audit log.
- Bulk activate or deactivate.
- Automatic or scheduled status changes.
- Email or other notifications.
- Pagination, server-side filtering, or server-side sorting.
- Password management changes.
- Changes to JWT secret, generation, or expiry settings.

6. Expected / Potentially Affected Files

Database

- `database/schema.sql`
- `database/seed.sql`
- `database/README.md`
- `database/migrations/CR-004_add_student_status.sql` (new)

- database/migrations/CR-001_rollback_student_status.sql (new)

Backend

- backend/src/models/studentModel.js
- backend/src/controllers/studentController.js
- backend/src/routes/studentRoutes.js
- backend/src/controllers/authController.js
- backend/src/models/userModel.js (optional)
- backend/src/middleware/authMiddleware.js (optional)

Frontend

- frontend/src/pages/DashboardPage.jsx
- frontend/src/pages/StudentDashboardPage.jsx
- frontend/src/components/StudentStatusBadge.jsx (optional)
- Existing dashboard stylesheet

Documentation

- CHANGELOG.md

Students may modify additional files if required. Any additional files must be identified and justified in the Implementation Report.

7. Functional, Database & Security Requirements

- FR-001: The administrator dashboard shall display the status (Active or Inactive) of every student.
- FR-002: An administrator shall be able to deactivate an active student and activate an inactive student.
- FR-003: The system shall ask the administrator to confirm before changing a student's status.
- FR-004: New students shall be created with ACTIVE status.
- FR-005: An inactive student shall not be able to log in and shall see a clear message that the account is inactive.
- FR-006: A student who is deactivated while logged in shall lose access on the next protected request and be redirected to the login page.
- FR-007: Activating a student again shall restore login access using the same credentials, without changing any student data.
- FR-008: Changing a status shall not modify or delete any other student or account data.
- FR-009: The dashboard shall provide an All/Active/Inactive status filter that works together with the CR-001 search, and the result count shall match the visible records.
- FR-010: Edit and Delete shall continue to use the correct `student_id`.
- DB-001: The `students.status` column shall accept only ACTIVE or INACTIVE and shall not allow NULL.
- DB-002: Existing student records shall be ACTIVE after migration, with no data loss.

- DB-003: A rollback script shall restore the previous structure, and `schema.sql` and `seed.sql` shall work on a fresh database.
- SEC-001: The status update endpoint shall require authentication and the ADMIN role. Students and unauthenticated users shall not be able to change any status.
- SEC-002: The inactive-student restrictions shall be enforced by the backend and not only by the frontend.
- SEC-003: The status value shall be validated on the server, parameterised queries shall be used, and no unsafe HTML rendering shall be introduced.
- SEC-004: The inactive-account message shall be shown only after the credentials are verified. The message for wrong credentials shall remain generic and unchanged.
- SEC-005: Existing authentication and authorization for all other endpoints, and the administrator login, shall remain unchanged.

8. Acceptance Criteria

The CR is successfully implemented when:

- AC-001: The `students` table contains the `status` column, existing students are ACTIVE after migration, and new students default to ACTIVE.
- AC-002: The administrator dashboard displays the status of every student.
- AC-003: An administrator can deactivate an active student after confirmation, and the new status is saved in the database and shown on the dashboard.
- AC-004: An inactive student cannot log in and sees a clear message, while active students can still log in.
- AC-005: A student who is deactivated while logged in is rejected on the next protected request and redirected to the login page.
- AC-006: An administrator can activate the student again, and the student can log in with the same credentials with no change to student data.
- AC-007: The status filter and the CR-001 search work together, and the result count matches the visible records.
- AC-008: Non-administrator and unauthenticated requests to change a status are rejected, and an invalid status value is rejected.
- AC-009: Existing Add, Edit, and Delete continue to affect the correct student, and the administrator login and authorization continue to work.
- AC-010: The migration script runs on an existing database without data loss, the rollback script restores the previous structure, and `schema.sql` and `seed.sql` work on a fresh database.

9. Expected Outcome

Administrators can deactivate and reactivate students without deleting their records. Inactive students are blocked from the system at login and with existing sessions, while the existing CRUD, search, authentication, and authorization functionality continues to work.

10. Implementation Report Requirement

After implementation, the Final Exam Software Change Implementation Report shall include:

- Implementation summary.
- Affected files.
- Important technical/security changes.
- Database migration and rollback verification.
- Test cases and results, including API (Postman) tests for the status endpoint and login restriction.
- Acceptance criteria verification.
- Relevant screenshots/evidence.
- Final status and outstanding issues.